MySQL Constraints Commands

Ensures that a column cannot have a NULL value.

```
CREATE TABLE table_name (
column_name datatype NOT NULL
);
```

Ensures that all values in a column are different.

```
CREATE TABLE table_name (
column_name datatype UNIQUE
);
```

To create a table with a column that has a UNIQUE constraint, you can use the UNIQUE keyword when defining the column. The UNIQUE constraint ensures that all values in the column are distinct. Here's an example that demonstrates creating a table with a UNIQUE constraint:

Example

create a table called Students where the Email column must have unique values.

```
CREATE TABLE Students (
    StudentID INT PRIMARY KEY,
    FirstName VARCHAR(50),
    LastName VARCHAR(50),
    Email VARCHAR(100) UNIQUE,
    EnrollmentDate DATE
);
```

Email: A variable-length string for the email, with a UNIQUE constraint to ensure no duplicate email addresses.

```
INSERT INTO Students (StudentID, FirstName, LastName, Email, EnrollmentDate) VALUES
(1, 'Alice', 'Johnson', 'alice.johnson@example.com', '2024-01-15'),
(2, 'Bob', 'Smith', 'bob.smith@example.com', '2024-01-16');
```

```
This will cause an error due to duplicate email
INSERT INTO Students (StudentID, FirstName, LastName, Email, EnrollmentDate)
VALUES (3, 'Charlie', 'Brown', 'alice.johnson@example.com', '2024-01-17');
```

A combination of a NOT NULL and UNIQUE. Uniquely identifies each row in a table.

```
CREATE TABLE table_name (
column_name datatype PRIMARY KEY
);
```

Ensures the referential integrity of the data in one table to match values in another table.

```
CREATE TABLE table_name (
    column_name datatype,
    FOREIGN KEY (column_name)
    REFERENCES
another_table(another_column)
);
```

In SQL, the FOREIGN KEY constraint is used to link two tables together. A foreign key in one table points to a primary key in another table. This enforces referential integrity by ensuring that a value in the foreign key column must exist in the referenced primary key column of the other table.

Example

Let's create two tables: Departments and Employees. The Employees table will have a foreign key that references the Departments table.

Create the Departments table

```
CREATE TABLE Departments (DeptID INT PRIMARY KEY, DeptName VARCHAR(100) NOT NULL);
```

Create the Employees table

```
CREATE TABLE Employees (EmployeeID INT PRIMARY KEY, FirstName VARCHAR(50), LastName VARCHAR(50), DeptID INT, FOREIGN KEY (DeptID) REFERENCES Departments(DeptID)
);
```

Employees Table:

•**DeptID**: An integer that acts as a foreign key, referencing the DeptID column in the Departments table. This ensures that any value in the DeptID column of the Employees table must exist in the DeptID column of the Departments table.

```
INSERT INTO Departments (DeptID, DeptName) VALUES
(1, 'HR'),
(2, 'IT'),
(3, 'Finance');


INSERT INTO Employees (EmployeeID, FirstName, LastName, DeptID) VALUES
(1, 'Alice', 'Johnson', 1),
(2, 'Bob', 'Smith', 2),
(3, 'Charlie', 'Brown', 3);
```

Attempting to Insert an Invalid Foreign Key

If you try to insert an employee with a DeptID that does not exist in the Departments table, it will result in an error:

This will cause an error because DeptID 4 does not exist in Departments

```
INSERT INTO Employees (EmployeeID, FirstName, LastName, DeptID) VALUES
(4, 'David', 'Williams', 4);
```

By using the FOREIGN KEY constraint, you ensure that the DeptID values in the Employees table are valid and correspond to existing DeptID values in the Departments table, thus maintaining referential integrity between the tables.

create table **department** (dept_name varchar (20), building varchar (15), budget numeric (12,2), primary key (dept_name));

create table **course** (course_id varchar(7), title varchar(50), dept_name varchar(20), credits numeric(2,0), primary key (course_id), foreign key(dept_name) references department(dept_name));

create table **instructor** (ID varchar (5), name varchar (20) not null, dept_name varchar (20), salary numeric(8,2), primary key (ID), foreign key(dept_name) references department(dept_name));

create table **section** (course_id varchar (8), sec_id varchar (8), semester varchar (6), year numeric(4,0), building varchar (15), room_number varchar (7), time_slot_id varchar (4), primary key (course_id, sec_id, semester, year), foreign key (course_id) references course(course_id));

create table **teaches** (ID varchar (5), course_id varchar (8), sec_id varchar (8), semester varchar(6), year numeric (4,0), primary key (ID, course_id, sec_id, semester, year), foreign key (course_id, sec_id, semester, year) references section(course_id, sec_id, semester, year), foreign key (ID) references instructor(ID));

create table **section** (course_id varchar (8), sec_id varchar (8), semester varchar (6), year numeric(4,0), building varchar (15), room_number varchar (7), time_slot_id varchar (4), primary key (course_id, sec_id, semester, year), foreign key (course_id) references course(course_id));

In the above schema definition for the section table, you are using a composite primary key. A composite primary key consists of two or more columns that together uniquely identify a row in the table. It is not that there are multiple primary keys, but rather that the primary key is composed of multiple columns.

Composite Primary Key:

- PRIMARY KEY (course_id, sec_id, semester, year) means that the combination of course_id, sec_id, semester, and year must be unique across the table. No two rows can have the same combination of these four values.

- This is useful for identifying each section of a course uniquely. For instance, a course might have different sections across different semesters and years.

INSERT INTO **course** (course_id, title, dept_name, credits) VALUES
('CS101', 'Introduction to Computer Science', 'Computer Science', 4), ('MATH101', 'Calculus I', 'Mathematics', 4);

INSERT INTO **section** (course_id, sec_id, semester, year, building, room_number, time_slot_id) VALUES
('CS101', '1', 'Fall', 2024, 'Main', '101', 'A1'),
('CS101', '2', 'Spring', 2025, 'Main', '102', 'B1'),
('MATH101', '1', 'Fall', 2024, 'Science', '201', 'C1');

This will cause an error because the combination (CS101, 1, Fall, 2024) already exists

```
INSERT INTO section (course_id, sec_id, semester, year, building, room_number, time_slot_id) VALUES
('CS101', '1', 'Fall', 2024, 'Main', '103', 'D1');
```

- A composite primary key uses multiple columns to ensure the uniqueness of each row in the table.
- This approach is beneficial when a single column cannot uniquely identify a row.
- The combination of the columns in the primary key must be unique for each row, thereby maintaining the integrity and uniqueness of the data in the table.

Ensures that the value in a column meets a specific condition.

```
CREATE TABLE table_name (
    column_name datatype,
    CHECK (condition)
);
```

Sets a default value for a column if no value is specified.

```
CREATE TABLE table_name (
column_name datatype DEFAULT default_value
);
```

In SQL, the DEFAULT constraint is used to provide a default value for a column. This default value is automatically assigned to the column when a new row is inserted without a specific value for that column.

Example

Let's create a table called Employees where the HireDate column has a default value of the current date, and the Department column has a default value of 'General'.

```
CREATE TABLE Employees ( EmployeeID INT PRIMARY KEY, FirstName VARCHAR(50), LastName VARCHAR(50), Department VARCHAR(50) DEFAULT 'General', HireDate DATE DEFAULT CURRENT_DATE );
```

Explanation

- **EmployeeID**: An integer primary key for unique identification of each employee.
- **FirstName**: A variable-length string for the first name.
- **LastName**: A variable-length string for the last name.
- **Department**: A variable-length string for the department, with a DEFAULT value of 'General'. If no value is provided for Department during insertion, 'General' will be used.
- **HireDate**: A date for the hire date, with a DEFAULT value of the current date. If no value is provided for HireDate during insertion, the current date will be used.

```
INSERT INTO Employees (EmployeeID, FirstName, LastName, Department, HireDate) VALUES
(1, 'Alice', 'Johnson', 'IT', '2024-01-15');
```

Insert without specifying Department and HireDate:

```
INSERT INTO Employees (EmployeeID, FirstName, LastName) VALUES
(2, 'Bob', 'Smith');

INSERT INTO Employees (EmployeeID, FirstName, LastName, Department) VALUES
(3, 'Charlie', 'Brown', 'HR');
```

Resulting Data

After the above insertions, the Employees table will look like this:

EmployeeID	FirstName	LastName	Department	HireDate
1	Alice	Johnson	IT	2024-01-15
2	Bob	Smith	**General**	2024-07-18 (assuming the current date is 2024-07-18)
3	Charlie	Brown	HR	2024-07-18 (assuming the current date is 2024-07-18)

MODIFY CONSTRAINT

```
ALTER TABLE table_name
MODIFY column_name datatype NOT NULL;
```

```
ALTER TABLE table_name
ADD UNIQUE (column_name);
```

MODIFY CONSTRAINT

```
ALTER TABLE table_name
ADD PRIMARY KEY (column_name);
```

```
ALTER TABLE table_name
ADD CONSTRAINT constraint_name
FOREIGN KEY (column_name)
REFERENCES
referenced_table(referenced_column);
```

```
ALTER TABLE table_name
ADD CHECK (condition);
```

```
ALTER TABLE table_name
ALTER COLUMN column_name SET DEFAULT
default_value;
```

MySQL Join Operations Commands

Selects records that have **matching values in both tables.**

```
SELECT columns
FROM table1
INNER JOIN table2 ON
table1.common_column =
table2.common_column;
```

Selects records that have matching values in both tables

Example:

```
SELECT employees.name,
departments.department_name
FROM employees
INNER JOIN departments ON
employees.department_id =
departments.department_id;
```

Selects all records from the left table, and the matched records from the right table. The result is NULL from the right side if there is no match

```
SELECT columns
FROM table1
LEFT JOIN table2 ON
table1.common_column =
table2.common_column;
```

Selects all records from the left table, and the matched records from the right table. The result is NULL from the right side if there is no match

Example:

```
SELECT employees.name,
departments.department_name
FROM employees
LEFT  JOIN  departments  ON
employees.department_id   =
departments.department_id;
```

Selects all records from the right table, and the matched records from the left table. The result is NULL from the left side if there is no match

```
SELECT columns
FROM table1
RIGHT JOIN table2 ON
table1.common_column =
table2.common_column;
```

Selects all records from the right table, and the matched records from the left table. The result is NULL from the left side if there is no match

Example:

```
SELECT employees.name,
departments.department_name
FROM employees
RIGHT JOIN departments ON
employees.department_id =
departments.department_id;
```

Selects all records when there is a match in either left or right table. Note that MySQL does not support FULL OUTER JOIN natively, but you can achieve this using a UNION of LEFT JOIN and RIGHT JOIN

```
SELECT columns
FROM table1
LEFT JOIN table2 ON table1.common_column = table2.common_column
UNION
SELECT columns
FROM table1
RIGHT JOIN table2 ON table1.common_column =
table2.common_column;
```

Selects all records when there is a match in either left or right table. Note that MySQL does not support FULL OUTER JOIN natively, but you can achieve this using a UNION of LEFT JOIN and RIGHT JOIN

Example:

```
SELECT employees.name, departments.department_name
FROM employees
LEFT JOIN departments ON employees.department_id =
departments.department_id
UNION
SELECT employees.name, departments.department_name
FROM employees
RIGHT JOIN departments ON employees.department_id =
departments.department_id;
```

Returns the Cartesian product of the two tables

```
SELECT columns
FROM table1
CROSS JOIN table2;
```

Example:

```
SELECT employees.name, departments.department_name
FROM employees
CROSS JOIN departments;
```